

Assessment 3 Report: Product Demonstration & Post-Implementation Review

PBT205 – Project-based Learning Studio: Technology

Submitted By:

Tia Darvell | A00029275

David Ristevski | A00072295

Submitted To:

Fahad Hameed

Torrens University Australia

Submission Date: 06 May 2026

Table of Contents

01 Background and Objectives	2
01.1 Scope of the Post-Implementation Review	3
01.2 Purpose of the Review	3
01.3 Original Project Objectives.....	3
02 Outcomes of Project Scope	3
02.1 Functional Requirements.....	3
02.2 Technical Requirements	5
03 Outcomes of Project Knowns and Unknowns	5
03.1 Assumptions.....	6
03.2 Risks	6
03.3 Constraints.....	7
04 Outcomes of Project Schedule.....	8
04.1 Timeline and Milestones	8
4.2 Deliverables	8
04.3 Resource Plan and Cost.....	9
05 Outcomes of Project Quality	9
05.1 Test Results	10
05.2 Defects and Resolutions.....	10
06 Group Reflection.....	10
06.1 What Went Well	10
06.2 What Could Be Improved	10
06.3 Lessons Learned – Technology	11
06.4 Lessons Learned – Team Dynamics	11
06.4 Overall Project Success Assessment.....	11
07 Conclusion	12
08 References	12

01 | Background and Objectives

01.1 | Scope of the Post-Implementation Review

This report is the post-implementation review of the extended XYZ Trading System, the prototype selected from Assessment 1 for further development in Assessment 2. The Chat Application and Contact Tracing System remain out of scope, as do the RabbitMQ middleware, serialisation contracts, and .NET 10 toolchain established in earlier assessments. The review covers work delivered between Module 7 and the Module 12 submission deadline.

01.2 | Purpose of the Review

The review reports project results against the Assessment 2 plan, determines whether stated objectives were met, captures lessons learned across technical and interpersonal dimensions, and accompanies a working demonstration of the extended system. It also serves as the formal closure document, recording deviations, defects, and any open items at submission.

01.3 | Original Project Objectives

The Assessment 2 initiation document defined three primary objectives: extend the single-stock trading system to support multiple concurrent stocks (FR-01); persist completed trades so the system survives restarts (FR-06); and replace the static single-stock GUI with a live multi-stock dashboard suitable for real-world trading visualisation (FR-05, FR-07). Two stretch objectives — visible order book (FR-08) and trading analytics (FR-09) — were classified as Could-have under MoSCoW prioritisation (Clegg & Barker, 1994), contingent on the Must and Should requirements being complete and integration-tested.

02 | Outcomes of Project Scope

02.1 | Functional Requirements

The brief defined nine functional requirements under the MoSCoW framework (Clegg & Barker, 1994). Five Must and two Should requirements were carried into Assessment 3 as in-scope deliverables; the two Could requirements were stretch items contingent on earlier work completing on schedule. Table 1 records delivered status with evidence references.

Table 1
Functional Requirements Delivery Status

ID	Requirement	Priority	Status	Evidence
FR-01	Multi-Stock Support	Must	Completed	tradingsystem.config.json, TradingCore/Configuration/StockConfig.cs, evidence/tc-06-per-stock-isolation.png
FR-02	Order Submission via CLI	Must	Completed	SendOrderApp/Program.cs (six-argument form with stock validation)
FR-03	Order Matching	Must	Completed	TradingCore/Services/OrderBookService.cs, evidence/tc-06-per-stock-isolation.png
FR-04	Trade Publication	Must	Completed	TradingCore/Services/RabbitMQService.cs, evidence/tc-06-rabbitmq-exchanges.png
FR-05	Live Price Feed	Must	Completed	evidence/stage-3-trade-broadcast.png, evidence/stage-3-browser-signalr.png, evidence/stage-4-dashboard-populated.png
FR-06	Persistent Trade History	Should	Completed	evidence/tc-09-restart-loaded.png (8 trades loaded from SQLite on restart)
FR-07	Trade History Display	Should	Completed	evidence/stage-4-dashboard-populated.png, evidence/stage-4-history-filter.png
FR-08	Order Book Visibility	Could	Not implemented	Optional: contingent on Should completion; deprioritised in favour of completing all Must and Should requirements
FR-09	Trading Analytics	Could	Not implemented	Same rationale as FR-08

Note. Priorities follow the MoSCoW framework defined in Assessment 2 04.1. Evidence file references correspond to images saved in evidence/ folder of the project repository and to test cases in evidence/test-results.md. Could requirements were classified in the Assessment 2 plan as stretch items contingent on Must and Should requirements being complete and integration tested.

All five Must requirements were delivered and verified end-to-end against a live RabbitMQ broker. Both Should requirements were also delivered: FR-06 was validated through the TC-09 restart-and-recover test, in which ExchangeApp was killed, restarted, and reported eight historical trades reloaded from SQLite; the dashboard correctly rendered pre-restart trades while the matching engine was offline. FR-07 surfaces those persisted trades through a scrollable, filterable history panel. The two Could requirements were deferred consistent with the Assessment 2 plan: although the Should requirements landed ahead of the Module 11 milestone, the team prioritised verification, documentation, and demonstration polish over commencing optional features.

02.2 | Technical Requirements

All seven technical requirements were implemented. RabbitMQ runs via Docker (Docker, Inc., 2024) with fanout exchanges declared at application start (VMware, 2024) (TR-01). The stack uses C# and .NET 10 (Microsoft, 2024a), ASP.NET Core for the GUI, and SignalR for real-time browser communication (Microsoft, 2024b) (TR-02). Message serialisation uses Newtonsoft.Json (Newton-King, 2024) with contracts defined in the shared TradingCore class library (TR-03). SQLite (SQLite Consortium, 2024) persistence is implemented through Entity Framework Core (Microsoft, 2024c), with the database context, entity models, and configurable database path managed within TradingCore (TR-04). The GUI is accessible from any modern browser at localhost (TR-05), and the full stack launches via a single docker compose up command (TR-06). Configuration values — including broker endpoint, stock list, and database path — are externalised to configuration files to eliminate hardcoded dependencies (TR-07). One documented deviation remains: the dashboard JavaScript hardcodes the visible stock list, marked inline as a TODO referencing a future GetStocks() hub call. The deviation is non-blocking and does not affect any validated functional behaviour.

Table 2
Technical Requirements Delivery Status

ID	Requirement	Status	Evidence
TR-01	RabbitMQ via Docker; fanout exchanges	Delivered	tc-06-rabbitmq-exchanges.png
TR-02	C# / .NET10; ASP.NET Core; SignalR	Delivered	*.csproj net10.0; stage-3-browser-signalr.png
TR-03	Newtonsoft.JSON; Trading class Library	Delivered	Shared models referenced by all four projects
TR-04	SQLite via Entity Framework Core	Delivered	tc-09-restart-loaded.png
TR-05	GUI accessible at localhost via browser	Delivered	stage-4-dashboard-empty.png
TR-06	Docker Compose orchestration	Delivered	evidence/tr-06-docker-compose-up.png, evidence/tr-06-docker-compose-trade.png — full stack verified from clean state; all three containers reached ready state in dependency order; end-to-end smoke test confirmed matched trade visible on dashboard
TR-07	Zero hardcoded values	Substantially Delivered	Endpoint, stocks, database path, quantity all configurable; one documented deviation in dashboard JavaScript

Note. Evidence file references correspond to images saved in the evidence/ folder. The TR-07 deviation is the dashboard's hardcoded stock list, documented inline at the call-site as a TODO referencing a future GetStocks() hub call; this deviation is non-blocking for any of the validated functional behaviours.

03 | Outcomes of Project Knowns and Unknowns

03.1 | Assumptions

The Assessment 2 plan documented seven assumptions made in the absence of explicit direction from the brief or learning facilitator. Six held in full; one — Assumption 5, fixed stock identifiers — held in spirit but was implemented more flexibly than originally envisaged through the configurable `tradingssystem.config.json` file. No assumption was contradicted in execution.

Table 3
Assumption Outcomes

#	Assumption	Outcome
1	RabbitMQ instance from A1 reused without modification	Held. Same broker configuration carried through all stages.
2	C# / .NET 10 remains the agreed development language	Held. All five projects target net10.0
3	Quantity fixed at 100 shares per order	Held. <code>SendOrderApp</code> validation rejects any non-100 quantity.
4	Demonstration video required for Assessment 3	Held. Demo video produced per Stage 6.3
5	Stock Identifiers fixed for the lifetime of an instance	Partially Held. Stock list is fixed at startup but is read from <code>tradingssystem.config.json</code> rather than baked in at compile time, allowing reconfiguration between runs.
6	Learning facilitator may alter or override technology decisions	Held. No facilitator changes occurred during the project.
7	No active contribution from the third team member should be assumed for planning purposes	Held – and proved Correct. Stages requiring the third members input were absorbed by the remaining team members or sequenced around their absence.

Note. Outcomes were assessed at the point of submission. Assumption 5 is recorded as partially held because the team's implementation introduced configurable stocks via `tradingssystem.config.json` (FR-01 / TR-07), which is more flexible than the original assumption envisaged but does not contradict it.

03.2 | Risks

Six project risks were registered in Assessment 2. Two materialised: the team-absence risk, which was the most consequential outcome of the project, and a minor SignalR JSON serialisation mismatch that surfaced during Stage 4 integration testing. Both were absorbed without impact to the submission deadline. The remaining four risks did not materialise. The team-absence mitigation — assigning no critical-path dependency to the third member and absorbing their work packages across the remaining two members — proved effective but substantially increased individual workload relative to the three-member plan.

Table 4
Risk Outcomes and Mitigation Effectiveness

Risk	Materialised	Mitigation Effective
Team Member absence reoccurs	Yes – high impact as forecast	Yes. Work packages were assigned with no critical-path dependency on the third member; remaining work was absorbed without missing the submission deadline.
Integration failures between independently developed	Partially – minor TradeHub serialisation mismatch surfaced during Stage 4 testing	Yes. Issue caught immediately but the integration-first testing approach and resolved by disabling the SignalR JSON PropertyNamingPolicy.
RabbitMQ.Client API changes in a future release	No	N/A - Dependency remained pinned at v7.2.1.
GUI performance degradation with multiple concurrent stocks	No	N/A -- Three stocks tested concurrently; no degradation observed
Scope creep from optional requirements	No	Yes. MoSCoW prioritization enforced; Could have remained deferred.
Docker or environment inconsistency across team machines.	No	Yes. Shared docker-compose.yml and tradingsystem.config.json ensured consistent local environments.

Note. The team-absence risk is recorded as the most consequential of the project. The mitigation strategy of front-loading work packaged with no dependency on the third member proved effective – the team did not miss any deliverables – but absorbed approximately 40% additional individual workload across Tia and David relative to a three-member assumption. This is reflected in 04.3 of this report.

03.3 | Constraints

The six constraints recorded in the Assessment 2 plan held throughout. The Module 12 deadline was met; the team size remained nominal at three with active capacity of two; RabbitMQ remained the prescribed middleware; C#/.NET 10 remained the development language; the GUI remained browser-accessible without additional end-user installation beyond Docker; and

configurable values remained externalised, with the single TR-07 deviation noted above. No constraint was relaxed, renegotiated, or breached.

04 | Outcomes of Project Schedule

04.1 | Timeline and Milestones

The Assessment 2 plan defined a six-week schedule from Module 7 to Module 12, with each module assigned a focused deliverable. All milestones were met on schedule. Table 5 records planned versus actual delivery.

Table 5
Schedule and Milestone Variance

Module	Planned Focus	Planned Milestone	Actual Outcome
7	Architecture and contracts	Message contracts frozen; Docker Compose verified	Completed on schedule. TradingCore models finalised; configuration design (tradingssystem.config.json) drafted.
8	Core CLI development	SendOrderApp and ExchangeApp passing integration tests	Completed on schedule. Stages 1.1, 1.3, 2.1, 2.3 delivered with TC-06 evidence captured.
9	Multi-stock extension persistence	FR-01 and FR-06 complete	Completed on schedule. SQLite layer (Stage 1.2 / 2.2) and unit tests (Stage 1.4) landed; TC-09 evidence captured.
10	GUI extensions	FR-05, FR-07 complete; live price feed and history visible	Completed on schedule. Stages 3.2, 3.3, 4.1–4.3, 4.5 delivered; per-stock dashboard verified end-to-end.
11	Integration and polish	All Must and Should requirements running concurrently	Completed on schedule. Demo evidence package captured; report drafting commenced.
12	Final Testing and Submission	All deliverables complete; demo recorded	Completed. Stage 6.1 verification finished. Demo video recorded. Submission package complete.

Note. Evidence file references correspond to images saved in the evidence/ folder. The TR-07 deviation is the dashboard's hardcoded stock list, documented inline at the call-site as a TODO referencing a future GetStocks() hub call; this deviation is non-blocking for any of the validated functional behaviours.

4.2 | Deliverables

Every deliverable defined in the Assessment 2 schedule was produced, with one qualification: the Could-have features (FR-08 and FR-09) were not built, consistent with their MoSCoW classification. All other deliverables — the docker-compose.yml, README, evidence folder, test results log, demonstration video, and this report — were produced as planned.

04.3 | Resource Plan and Cost

The Assessment 2 plan budgeted 51 hours of effort at a notional junior developer rate of \$50 per hour, totalling \$2,550. Hours were not formally tracked across the project; the auditable record of contribution is the project's GitHub commit history at <https://github.com/daveinciii/pbt205-xyz-exchange>.

Table 6
Repository Contributions per Team Member

Member	Commits	Additions	Deletions	Active Period
Tia Darvell	12	3,010	1,322	March – May 2026
David Ristevski	10	16, 935	335	March – May 2026
Nicholas Beltran	0	0	0	-
Total	22	19, 945	1,657	

Note. Data captured from the project's Contributors graph at <https://github.com/daveinciii/pbt205-xyz-exchange/graphs/contributors> and Pulse view at <https://github.com/daveinciii/pbt205-xyz-exchange/pulse> as of 6 May 2026. The full per-commit, timestamped record is available at <https://github.com/daveinciii/pbt205-xyz-exchange/commits/main/>. Commit counts reflect work pushed to the main branch excluding merge commits. Line totals include auto-generated content (EF Core migration files, Docker scaffolding, NuGet lockfiles) and are therefore an imprecise proxy for productive effort; the commit count and per-stage commit messages provide a more faithful indicator of contribution. Nicholas Beltran's row of zero contributions reflects the team-absence outcome documented in §03.2 of this report.

Two team members contributed substantively. Tia delivered twelve commits centred on Stages 1.1, 1.3, 2.1, 2.3, 3.2, 4.1–4.3, 4.5, and the report itself; David delivered ten commits centred on Stages 1.2, 1.4, 2.2, 5.1, and 5.2 — primarily the SQLite persistence layer, unit testing, Docker Compose, and Dockerfile authoring. Effort was distributed unevenly relative to the original resource plan: the third member's allocated work was absorbed by Tia and David through reassignment, in line with the contingency built into the plan. The notional cost figure of \$2,550 is unchanged and remains intended for comparative purposes only.

05 | Outcomes of Project Quality

05.1 | Test Results

Persistence functionality was validated through the TC-09 restart-and-recover scenario, which confirmed that trade data was written to SQLite and reloaded correctly after both local-process and Docker-container restarts; the Docker case verified that the volume mount preserved the database file across container resets. Unit testing covered the OrderBookService matching logic, including unmatched orders, conditional trade execution, and per-stock isolation. All unit tests pass under the .NET test runner. The full stack was verified end-to-end from a clean Docker state through to a matched trade visible on the dashboard, and the codebase builds without errors or warnings.

05.2 | Defects and Resolutions

Two defects were identified and resolved during development. The first was the loss of trade history when the application restarted under in-memory storage; this was resolved by introducing SQLite persistence via Entity Framework Core. The second was a connection-ordering issue between the TradeListenerService and the GUI when both started concurrently in Docker — the listener attempted to connect before RabbitMQ was ready. This was resolved by implementing an exponential backoff and retry strategy in the listener, allowing it to reach a ready state without blocking the GUI's startup.

06 | Group Reflection

06.1 | What Went Well

The architectural decision to centre the system on RabbitMQ fanout exchanges proved to be the most consequential technical success of the project. Fanout was chosen over direct or topic exchanges because every consumer — the order book, GUI, and any future analytics subscriber — needs the full message stream without filtering. Adding new consumers therefore required no changes to exchange configuration or publishing code, which allowed the GUI and persistence layer to be developed and integrated independently. Centralising shared models, serialisation, and configuration in the TradingCore class library reinforced this gain: every project referenced a single source of truth for message contracts, eliminating an entire category of integration defect.

The MoSCoW prioritisation framework held without renegotiation. Deferring FR-08 and FR-09 was a deliberate strategy that preserved capacity to deliver the Must and Should requirements to a high standard rather than spreading effort across seven features at varying levels of completeness. The decision proved correct: all five Must requirements and both Should requirements were delivered, verified, and evidenced before the Module 11 milestone.

06.2 | What Could Be Improved

The most significant process improvement available would have been earlier and more granular integration testing. Although Assessment 2 identified integration as a key risk and the team adopted module-boundary checkpoints, integration testing in early stages was effectively deferred

until a feature was considered individually complete. A minimal integration harness defined at the start of Stage 3 — against which both the GUI and exchange backend were tested continuously rather than at the point of first connection — would have caught the SignalR serialisation defect even earlier than it was. Unit test coverage was also narrower than ideal: six tests covered the primary matching scenarios but did not extend to partial fills, boundary price equality, or concurrent submission under load. The trade-off was defensible given the two-member capacity constraint, but a broader suite would have provided stronger regression protection during the multi-stock extension.

06.3 | Lessons Learned – Technology

This project highlighted the importance of designing for scalability and extensibility from the outset. Moving from a single-stock to a multi-stock architecture required no structural rewrite because hardcoded values had been replaced with externalised configuration and shared services in the TradingCore library; new stock symbols could be supported by editing a configuration file rather than modifying compiled code. RabbitMQ provided clean separation between subsystems — order submission, matching, and GUI updates ran as independent processes communicating through exchanges (VMware, 2024), which made each subsystem testable in isolation and contained failures to a single component. Adopting SQLite (SQLite Consortium, 2024) with Entity Framework Core (Microsoft, 2024c) taught the team to treat persistence as a first-class architectural concern rather than an in-memory afterthought. Finally, SignalR (Microsoft, 2024b) demonstrated how an event-driven, push-based model removes the need for client-side polling and keeps GUI state synchronised with backend events in real time.

06.4 | Lessons Learned – Team Dynamics

Operating with two active members rather than the planned three required the team to broaden each member's role across development, testing, and documentation. The most important lesson was that team resilience is a planning output, not a response to adversity: the strategies that allowed the project to succeed with two members — assigning no critical-path dependency to the absent member, distributing work packages between Tia and David, and maintaining a shared docker-compose.yml and configuration file so both members worked against identical environments — were designed into the Assessment 2 plan before the absence materialised, not improvised after it.

The two active members also implemented a lightweight integration checkpoint at the start of each module: before commencing new stage work, the shared branch was pulled, built, and smoke-tested to confirm the integration baseline was stable. This caught the SignalR serialisation mismatch at the earliest possible point. Communication discipline — short asynchronous status updates tied to commit messages rather than scheduled meetings — kept both members aware of interdependencies without imposing meeting overhead that the reduced team could not absorb. Shared ownership of the test suite and the evidence folder ensured that quality artefacts were a team output rather than one member's responsibility.

06.4 | Overall Project Success Assessment

The project is assessed as successful against its stated objectives. All five Must requirements (FR-01 to FR-05) and both Should requirements (FR-06 and FR-07) were implemented, verified end-to-

end, and evidenced. The system supports multiple concurrent stocks, persists trade history across restarts, and surfaces a real-time multi-stock dashboard — a substantive extension of the Assessment 1 prototype. The two Could requirements were deferred in accordance with their MoSCoW classification, which always made them contingent on the prior work being complete and integration tested. From a project-management perspective, every milestone was met on schedule and no constraint was breached. The codebase builds cleanly, all unit tests pass, and the full stack runs from a single docker compose up command. The project is submitted as a complete and coherent deliverable.

07 | Conclusion

The XYZ Trading System delivered for Assessment 3 is a substantive extension of the Assessment 1 prototype. The system accepts orders from multiple independent CLI clients, matches them against isolated per-stock order books, persists every completed trade to a SQLite database that survives process restarts, and broadcasts live updates to a multi-stock dashboard over SignalR — all orchestrated by a single docker compose up command. Every Must and Should functional requirement was implemented, verified against a live RabbitMQ broker, and evidenced. The two Could requirements were deferred in line with their MoSCoW classification.

The project was completed by two of the three enrolled members. The contingency strategy documented in Assessment 2 — assigning no critical-path dependency to the absent member — proved effective: no milestone was missed and no requirement was descoped. Technically, the project demonstrated that a message-driven, loosely coupled architecture absorbs scope more gracefully than a tightly integrated one; adding multi-stock support, persistence, and a live dashboard each required only additive changes rather than structural rewrites. The system is stable, fully documented, and ready for demonstration.

08 | References

Clegg, D., & Barker, R. (1994). *Case method fast-track: A RAD approach*. Addison-Wesley.

Docker, Inc. (2024). *Docker Compose documentation*. <https://docs.docker.com/compose/>

Microsoft. (2024a). *.NET documentation*. Microsoft Learn. <https://learn.microsoft.com/en-us/dotnet/>

Microsoft. (2024b). *ASP.NET Core SignalR documentation*. Microsoft Learn. <https://learn.microsoft.com/en-us/aspnet/core/signalr/>

Microsoft. (2024c). *Entity Framework Core documentation*. Microsoft Learn. <https://learn.microsoft.com/en-us/ef/core/>

Newton-King, J. (2024). *Json.NET documentation*. Newtonsoft. <https://www.newtonsoft.com/json/help/html/Introduction.htm>

SQLite Consortium. (2024). *SQLite documentation*. <https://www.sqlite.org/docs.html>

VMware. (2024). *RabbitMQ documentation*. <https://www.rabbitmq.com/docs>